

C867

# Scripting and Programming

Armored Drill

8 concepts · 12 items · 4 transfer items

**What this is.** Every concept in this course's public description, re-asked with more disguise than a practice test uses. Each stem carries facts that do not decide it. Every wrong answer is a named trap.

**Source.** Scope from the course's public description in the WGU Institutional Catalog (September 2026); items written from the field's standard body of knowledge. No WGU assessment item, course material or proprietary content is used or reproduced. Not affiliated with or endorsed by Western Governors University.

Method © 2026 Erik Daniel Lindquist — Course Mastery Coach, armored drill. Items drafted with AI assistance under his direction.

# How to run it

Paper first. Pencil. One pass, no notes, a timer if you want the pressure.

1. **Before you read the options**, fill the strip under the item: the target word (what is actually being asked), and the one fact in the stem that decides it.
2. **Principle before arithmetic**. Name the rule, then compute.
3. **Camouflage**. Most of every stem does not matter. Ask: which single fact decides this?
4. **Inverse pairs are the bait**. If two options are opposites, one of them is the trap built for you.
5. **Mark sure, unsure or guess** honestly. Sure-and-wrong is the most valuable miss on the page — it is a wrong mental model.
6. **Absolutes** (always, never, guarantee) are usually wrong. And recognizing an answer is not the same as owning it.
7. Items with a **heavy black bar** on the left are **transfer items**: the same principle, reversed or solved for a different variable.
8. Stop at the STOP page. Then grade, fill the triage grid, and tally the traps you fell for. The biggest tally is what you drill next.

The strip, filled in, looks like this:

|                             |                                           |                        |                                  |
|-----------------------------|-------------------------------------------|------------------------|----------------------------------|
| <b>Target word</b><br>_____ | <b>The one fact that decides</b><br>_____ | <b>Answer</b><br>_____ | [ ] sure [ ] unsure [ ]<br>guess |
|-----------------------------|-------------------------------------------|------------------------|----------------------------------|

# Items

1.

A warehouse program written by a team of four declares `int boxes = 17;` and `int perPallet = 5;` then prints `boxes / perPallet`. The code compiles with no warnings on the team's build server. What is printed?

- A. 3
- B. 2
- C. 4
- D. 3.4

| Target word | The one fact that decides | Answer | [ ] sure [ ] unsure [ ] guess |
|-------------|---------------------------|--------|-------------------------------|
| _____       | _____                     | ___    |                               |

2. [TRANSFER — reversed stem / different variable]

Which expression evaluates to 2?

- A. `17 / 5`
- B. `17 % 5`
- C. `17.0 / 5`
- D. `5 % 17`

| Target word | The one fact that decides | Answer | [ ] sure [ ] unsure [ ] guess |
|-------------|---------------------------|--------|-------------------------------|
| _____       | _____                     | ___    |                               |

3.

A payroll tool has `void addBonus(int pay) { pay = pay + 500; }`. In main, `int salary = 40000; addBonus(salary); cout << salary;` runs. The function was reviewed by a senior developer, and bonuses are paid in December. What is printed?

- A. 40000
- B. A compile error, because void functions can never change parameters.
- C. 500
- D. 40500

| Target word | The one fact that decides | Answer | [ ] sure [ ] unsure [ ] guess |
|-------------|---------------------------|--------|-------------------------------|
| _____       | _____                     | ___    |                               |

**4.** [TRANSFER — reversed stem / different variable]

Using the same addBonus function, which single change to its parameter list makes salary print 40500 without changing the call?

- A.** Change int pay to int\* pay.
- B.** Change int pay to double pay.
- C.** Change int pay to const int& pay.
- D.** Change int pay to int& pay.

| Target word | The one fact that decides | Answer | [ ] sure [ ] unsure [ ] guess |
|-------------|---------------------------|--------|-------------------------------|
| _____       | _____                     | ___    |                               |

**5.**

A grading program stores 10 quiz scores in int scores[10]. The instructor teaches 32 students and uses the program every Friday. Which expression reads the last quiz score?

- A.** scores[-1]
- B.** scores[32]
- C.** scores[9]
- D.** scores[10]

| Target word | The one fact that decides | Answer | [ ] sure [ ] unsure [ ] guess |
|-------------|---------------------------|--------|-------------------------------|
| _____       | _____                     | ___    |                               |

**6.** [TRANSFER — reversed stem / different variable]

An array's last valid index is 49. How many elements does it hold?

- A.** 48
- B.** It depends on the size of each element in bytes.
- C.** 50
- D.** 49

| Target word | The one fact that decides | Answer | [ ] sure [ ] unsure [ ] guess |
|-------------|---------------------------|--------|-------------------------------|
| _____       | _____                     | ___    |                               |

**7.**

In a thermostat controller written for a client in Denver, int temp = 8; int\* p = &temp; \*p = 20; cout << temp; runs on a board with 256 KB of memory. What is printed?

- A.** 28
- B.** 8
- C.** The memory address of temp
- D.** 20

| Target word | The one fact that decides | Answer | [ ] sure [ ] unsure [ ] guess |
|-------------|---------------------------|--------|-------------------------------|
| _____       | _____                     | ___    |                               |

8.

A class written for a regional credit union has private: double balance; and public: void deposit(double amt); double getBalance();. In main, a new account starts at \$0 and the code runs acct.balance = 100;. What happens?

- A. The program compiles but crashes at run time.
- B. It works, since any code can always set any member of an object it created.
- C. A compile error, because balance is private.
- D. balance is set to 100.

| Target word | The one fact that decides | Answer | [ ] sure [ ] unsure [ ] guess |
|-------------|---------------------------|--------|-------------------------------|
| _____       | _____                     | ___    |                               |

9.

A shipping app handles an order of 10 items for a customer in Ohio. It runs for (int i = 1; i <= 10; i += 2) { printLabel(i); }. How many times is printLabel called?

- A. 5
- B. 9
- C. 6
- D. 10

| Target word | The one fact that decides | Answer | [ ] sure [ ] unsure [ ] guess |
|-------------|---------------------------|--------|-------------------------------|
| _____       | _____                     | ___    |                               |

10. [TRANSFER — reversed stem / different variable]

The loop for (int i = 0; i <= n; i++) runs exactly 12 times. What is n?

- A. 12
- B. 11
- C. 0
- D. 13

| Target word | The one fact that decides | Answer | [ ] sure [ ] unsure [ ] guess |
|-------------|---------------------------|--------|-------------------------------|
| _____       | _____                     | ___    |                               |

11.

A graphics library licensed from a vendor defines class Shape { public: double area(); }; and class Circle : public Shape that defines its own area(). The code runs Shape\* s = new Circle(); s->area(); on a 64-bit build. Which area() runs?

- A. Circle::area(), because the object is a Circle.
- B. Shape::area(), because the function is not virtual.
- C. A compile error, because Circle redefines area().
- D. Both functions run, base first.

| Target word | The one fact that decides | Answer | [ ] sure [ ] unsure [ ] guess |
|-------------|---------------------------|--------|-------------------------------|
| _____       | _____                     | ___    |                               |

12.

A function written by a contractor and compiled at -O2 runs each time a sensor reading arrives. It does `int* buf = new int[1000]`; fills `buf`, computes an average, and returns the average. After a day the process uses several gigabytes. What is the problem and fix?

- A. Nothing; C++ frees heap memory when the function returns.
- B. A stack overflow; make the array smaller.
- C. A memory leak; call `delete buf` before returning.
- D. A memory leak; call `delete[] buf` before returning.

| Target word | The one fact that decides | Answer | <input type="checkbox"/> sure <input type="checkbox"/> unsure <input type="checkbox"/> guess |
|-------------|---------------------------|--------|----------------------------------------------------------------------------------------------|
| _____       | _____                     | ____   |                                                                                              |

# STOP

Answers and dissection follow. Finish every item and every strip first.

# Key and dissection

Struck-through text is the camouflage: facts that do not decide the item.

## 1. Answer A Concept 1

A warehouse program ~~written by a team of four~~ declares `int boxes = 17;` and `int perPallet = 5;` then prints `boxes / perPallet`. ~~The code compiles with no warnings on the team's build server.~~ What is printed?

**Target word:** output of `int / int` **Decides:** Both operands are `int`, so the result truncates.

|   |                    |                                                              |
|---|--------------------|--------------------------------------------------------------|
| A | <b>Correct</b>     | <code>17 / 5</code> in integer math is 3; the .4 is dropped. |
| B | Confusion set      | That is <code>17 % 5</code> , the remainder.                 |
| C | Inverted operation | Rounds up instead of truncating.                             |
| D | Adjacent category  | That is floating-point division.                             |

## 2. Answer B Concept 1 · transfer

Which expression evaluates to 2?

**Target word:** pick the expression from the result **Decides:** 2 is the remainder of 17 divided by 5.

|   |                    |                                                      |
|---|--------------------|------------------------------------------------------|
| A | Confusion set      | Integer quotient, which is 3.                        |
| B | <b>Correct</b>     | <code>17 = 3 × 5 + 2</code> , so the remainder is 2. |
| C | Adjacent category  | Floating-point division gives 3.4.                   |
| D | Inverted operation | Swapped operands give 5.                             |

## 3. Answer A Concept 2

A payroll tool has `void addBonus(int pay) { pay = pay + 500; }`. In main, `int salary = 40000; addBonus(salary);` `cout << salary;` runs. ~~The function was reviewed by a senior developer, and bonuses are paid in December.~~ What is printed?

**Target word:** caller's value after a by-value call **Decides:** `pay` is a copy of `salary`.

|   |                   |                                                                |
|---|-------------------|----------------------------------------------------------------|
| A | <b>Correct</b>    | Only the copy changed; <code>salary</code> is untouched.       |
| B | Absolute language | It compiles; void functions can change reference parameters.   |
| C | Wrong stem number | The bonus amount, not <code>salary</code> .                    |
| D | Confusion set     | That would need <code>int&amp; pay</code> (pass by reference). |

## 4. Answer D Concept 2 · transfer

Using the same `addBonus` function, which single change to its parameter list makes `salary` print 40500 without changing the call?

**Target word:** make the change reach the caller **Decides:** A reference parameter aliases the caller's variable.

|   |                    |                                                                                              |
|---|--------------------|----------------------------------------------------------------------------------------------|
| A | Dropped element    | A pointer also needs <code>&amp;salary</code> in the call and <code>*pay</code> in the body. |
| B | Adjacent category  | A different type is still a copy.                                                            |
| C | Inverted operation | <code>const</code> forbids changing the value.                                               |
| D | <b>Correct</b>     | A reference parameter modifies <code>salary</code> directly.                                 |

## 5. Answer C Concept 3

A grading program stores 10 quiz scores in `int scores[10]`. ~~The instructor teaches 32 students and uses the program every Friday.~~ Which expression reads the last quiz score?

**Target word:** index of last element **Decides:** Valid indexes are 0 to 9.

|   |                   |                                                             |
|---|-------------------|-------------------------------------------------------------|
| A | Confusion set     | Python-style negative indexing does not work on C++ arrays. |
| B | Camouflage        | Uses the class size, which is not the array size.           |
| C | <b>Correct</b>    | 10 elements run from index 0 to 9.                          |
| D | Wrong stem number | One past the end; undefined behavior.                       |

## 6. Answer C Concept 3 · transfer

An array's last valid index is 49. How many elements does it hold?

**Target word:** solve for size from last index **Decides:** Size = last index + 1.

|   |                    |                                                   |
|---|--------------------|---------------------------------------------------|
| A | Inverted operation | Subtracts one instead of adding.                  |
| B | Adjacent category  | Byte size affects memory used, not element count. |
| C | <b>Correct</b>     | Indexes 0 through 49 are 50 slots.                |
| D | Dropped element    | Forgets index 0.                                  |

## 7. Answer D Concept 4

In a thermostat controller ~~written for a client in Denver~~, `int temp = 8; int* p = &temp; *p = 20; cout << temp;` runs ~~on a board with 256 KB of memory~~. What is printed?

**Target word:** value after write through a pointer **Decides:** `*p = 20` writes to `temp`'s memory.

|   |                    |                                                                                                  |
|---|--------------------|--------------------------------------------------------------------------------------------------|
| A | Inverted operation | Treats <code>=</code> as <code>+=</code> .                                                       |
| B | Confusion set      | Treats <code>p</code> as holding a separate copy.                                                |
| C | Adjacent category  | That would be <code>cout &lt;&lt; p</code> or <code>&amp;temp</code> .                           |
| D | <b>Correct</b>     | <code>p</code> points to <code>temp</code> , so <code>*p = 20</code> changes <code>temp</code> . |

## 8. Answer C Concept 5

A class ~~written for a regional credit union~~ has `private: double balance;` and `public: void deposit(double amt); double getBalance();`. In main, ~~a new account starts at \$0 and~~ the code runs `acct.balance = 100;`. What happens?

**Target word:** outside access to a private member **Decides:** `balance` is private.

|   |                   |                                                             |
|---|-------------------|-------------------------------------------------------------|
| A | Adjacent category | Access rules are checked at compile time.                   |
| B | Absolute language | Creating an object grants no access to its private members. |
| C | <b>Correct</b>    | Only member functions can access private members.           |
| D | Confusion set     | Treats the member as public.                                |

## 9. Answer A Concept 6

A shipping app handles an order of 10 items ~~for a customer in Ohio~~. It runs for (int i = 1; i <= 10; i += 2) { printLabel(i); }. How many times is printLabel called?

**Target word:** iteration count **Decides:** i takes 1, 3, 5, 7, 9.

|   |                   |                                                        |
|---|-------------------|--------------------------------------------------------|
| A | <b>Correct</b>    | i = 1, 3, 5, 7, 9; at 11 the test fails.               |
| B | Confusion set     | That is the last value of i, not the number of passes. |
| C | Dropped element   | Counts i = 11, where the test has already failed.      |
| D | Wrong stem number | Reads the bound as the count, ignoring the step.       |

## 10. Answer B Concept 6 · transfer

The loop for (int i = 0; i <= n; i++) runs exactly 12 times. What is n?

**Target word:** solve for bound **Decides:** i runs 0 through n, which is n + 1 passes.

|   |                    |                                         |
|---|--------------------|-----------------------------------------|
| A | Dropped element    | Ignores the = in <=, which adds a pass. |
| B | <b>Correct</b>     | 0 through 11 is 12 values.              |
| C | Wrong stem number  | That is the start value.                |
| D | Inverted operation | Adds one instead of subtracting.        |

## 11. Answer B Concept 7

A graphics library ~~licensed from a vendor~~ defines class Shape { public: double area(); }; and class Circle : public Shape that defines its own area(). The code runs Shape\* s = new Circle(); s->area(); ~~on a 64 bit build~~. Which area() runs?

**Target word:** which function is called **Decides:** Shape::area is not virtual.

|   |                   |                                              |
|---|-------------------|----------------------------------------------|
| A | Confusion set     | True only if Shape::area were virtual.       |
| B | <b>Correct</b>    | Without virtual, the pointer's type decides. |
| C | Adjacent category | Hiding a base function is legal.             |
| D | Dropped element   | That is constructor order, not method calls. |

## 12. Answer D Concept 8

A function ~~written by a contractor and compiled at O2~~ runs each time a sensor reading arrives. It does int\* buf = new int[1000]; fills buf, computes an average, and returns the average. After a day the process uses several gigabytes. What is the problem and fix?

**Target word:** diagnose and fix **Decides:** new[] with no delete[].

|   |                    |                                                                             |
|---|--------------------|-----------------------------------------------------------------------------|
| A | Confusion set      | Only stack variables are freed automatically; C++ has no garbage collector. |
| B | Adjacent category  | new allocates on the heap, not the stack.                                   |
| C | Inverted operation | Arrays from new[] must be released with delete[].                           |
| D | <b>Correct</b>     | Heap memory is never freed without delete[].                                |

## Triage grid

Write each item number in the box it belongs to.

|                       | Right                                                 | Wrong                                                                                 |
|-----------------------|-------------------------------------------------------|---------------------------------------------------------------------------------------|
| <b>Sure</b>           | Owned.                                                | <b>Concept inversion — top priority.</b> A wrong mental model; it survives rewording. |
| <b>Unsure / guess</b> | Recognition, not mastery. Re-drill with the transfer. | Not learned yet.                                                                      |

## Trap tally

For every miss, find the letter you chose in the key and tally its trap. The biggest number is the pattern to drill.

| Trap               | What it does                                                 | In this drill | Your misses |
|--------------------|--------------------------------------------------------------|---------------|-------------|
| Dropped element    | Leaves out a required term or step.                          | 5             |             |
| Wrong stem number  | A figure copied from the prompt that is not the answer.      | 4             |             |
| Adjacent category  | The right idea in the wrong bucket.                          | 8             |             |
| Inverted operation | Direction flipped: add/subtract, cause/effect, higher/lower. | 7             |             |
| Absolute language  | Always, never, guarantee — where judgment applies.           | 2             |             |
| Confusion set      | Every option is real; the test is exact role-matching.       | 9             |             |
| Camouflage         | Answers a stem fact that was never the question.             | 1             |             |

# Anchor rows — the concept list

One row per concept: the rule to lock in, the trap that catches people, and the tell in the question that signals it.

| # | Concept                                    | Rule                                                                                                           | Trap                                                                | Tell                                                                              |
|---|--------------------------------------------|----------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------|-----------------------------------------------------------------------------------|
| 1 | <b>Integer division and modulus</b>        | When both operands are int, / truncates toward zero and % gives the remainder.                                 | Expecting 17 / 5 to give 3.4.                                       | Two int variables divided and printed.                                            |
| 2 | <b>Pass by value vs. by reference</b>      | A value parameter is a copy; only a reference (&) or pointer parameter can change the caller's variable.       | Thinking a function changes any variable passed into it.            | A void function that modifies its parameter, then the caller prints the original. |
| 3 | <b>Zero-based array indexing</b>           | An array of n elements has valid indexes 0 through n – 1.                                                      | Using arr[n] as the last element.                                   | "Last element" or a loop bound on an array.                                       |
| 4 | <b>Pointers: &amp; and *</b>               | &x gives x's address; *p reads or writes the value at the address p holds.                                     | Thinking a pointer holds its own copy of the value.                 | An assignment through *p, then the original variable is printed.                  |
| 5 | <b>Encapsulation and access specifiers</b> | Private members can be reached only by the class's own member functions; outside code must use public methods. | Thinking code that owns an object can set its private fields.       | obj.field = value where field is private.                                         |
| 6 | <b>Loop iteration counts</b>               | Count the values the loop variable actually takes: start, test, and step together decide it.                   | Reading the bound number as the count, or ignoring <= vs. <.        | A for loop with an unusual start, step, or <=.                                    |
| 7 | <b>Virtual functions and polymorphism</b>  | Through a base-class pointer, the derived override runs only if the base function is declared virtual.         | Assuming the object's real type always decides which function runs. | Base* p = new Derived; p->f(); with or without virtual.                           |
| 8 | <b>Dynamic memory and leaks</b>            | Every new needs a matching delete (new[] with delete[]); C++ has no garbage collector.                         | Assuming memory frees itself when a function returns.               | new inside a function that is called repeatedly with no delete.                   |

**Course scope (public description):** Applications - In this undergraduate course students explore the various aspects of the C++ programming language by examining its syntax, the development environment, and tools and techniques to solve some real-world problems.